학습 목표

도구를 모델용 인터페이스로 설계하고 공통 실행기로 묶는다

1. **ACI 원칙에 따라 API 래퍼형 도구를 워크플로 단위 도구로 재설계할 수 있다**
   도구 6개를 3개로 줄이는 예제로 연습

2. **@beta_tool과 strict 스키마로 도구를 정의하고 pydantic으로 입력을 재검증할 수 있다**
   strict: true와 additionalProperties: false 적용

3. **@tool과 create_sdk_mcp_server로 인프로세스 MCP 도구 서버를 만들 수 있다**
   readOnlyHint로 병렬 실행 대상 표시

4. **타임아웃, 에러 변환, 출력 오프로딩, 감사 로그를 갖춘 공통 실행기를 구현할 수 있다**
   모든 도구 호출이 거쳐 가는 execute_tool()

5. **모델이 읽고 스스로 고칠 수 있는 에러 메시지를 작성할 수 있다**

무단전재 및 배포 금지

도입 사례: 잘못 쓰기 쉬운 도구

'123번 티켓 해결 처리' 요청의 실행 기록(설명용 가상 예시)

text

```
[turn 1] list_tickets({})
  → 티켓 전체 JSON, 수만 토큰
[turn 2] get_ticket({"id": "123"})
  → "KeyError: '123'"
[turn 3] raw_sql("SELECT * ... WHERE id LIKE '%123%'")
  → 3건
[turn 4] list_docs({})
  → 문서 40개 본문 전체, 다시 수만 토큰
[turn 5] raw_sql("UPDATE tickets SET status='closed'
                  WHERE id LIKE '%123%'")
  → 3 rows affected
```

1. 전체 목록 도구가 컨텍스트를 수만 토큰씩 낭비함
2. **id**라는 모호한 파라미터는 넣을 형식을 알려 주지 않음
3. 파이썬 예외 문자열은 다음에 할 일을 말해 주지 않음
4. raw_sql이 티켓 세 건을 한꺼번에 닫는 사고를 막지 못함

> ***실패 원인은 도구 인터페이스다: ACI(Agent-Computer Interface)에 HCI만큼 공을 들여라***

출처: Anthropic Engineering, "Building effective agents" (2024-12-19)

무단전재 및 배포 금지

ACI 설계 원칙 여섯 가지

도구는 모델을 위한 사용자 인터페이스다

설명 문서

- 신입 개발자용 문서처럼 작성
- 무엇을, 언제 쓰나와 쓰지 않나
- 파라미터 예시와 반환 형태

적은 도구

- 워크플로 단위, list_all 대신 search
- 기능이 겹치지 않게 자기완결적으로
- 정의 토큰은 매 요청 포함, count_tokens로 측정
- 정의가 바뀌면 프롬프트 캐시 무효화

이름 짓기

- 서비스_리소스_동작 형태
- id 대신 ticket_id
- 반환은 사람이 읽는 필드로

응답 크기

- 전체 목록 대신 페이지네이션
- response_format으로 상세도 선택
- 기본 응답이 실행기 상한보다 작게
- 자세한 설계는 다음 슬라이드

에러 메시지

- 무엇이 틀렸는지
- 어떻게 고치는지
- 못 고치면 무엇을 할지

poka-yoke

- 애초에 틀릴 수 없게 만들기
- 예: 절대경로만 받는 도구
- 예: closed를 enum에서 제외

> ***사람도 헷갈리는 도구 선택은 에이전트도 헷갈린다***

출처: Anthropic Engineering, "Writing effective tools for agents" (2025-09-11), "Effective context engineering for AI agents" (2025-09-29)

무단전재 및 배포 금지

토큰 효율적인 응답 설계

도구의 기본 응답 자체를 실행기 상한보다 작게 만든다

응답 크기를 다루는 장치

장치	구현	효과
response_format	concise 또는 detailed	필요한 상세도를 모델이 고름
대상 지정 조회	doc_ids 최대 2개	훑기 후 골라 읽기 지원
페이지네이션	page, page_size, has_more	필요한 만큼만 더 가져옴
잘라내기 기본값	detailed 2건, 발췌 약 2,400B	기본 응답이 상한보다 작음
출력 크기 상한	실행기 max_bytes 4KB	구현 실수를 막는 최후 방어선

1. 잘라내기 기본값과 출력 크기 상한은 역할이 다름
2. 정상 호출된 도구는 실행기 상한에 걸리지 않아야 함
3. 상한에 걸렸다면 도구 설계가 잘못된 것으로 봄

무단전재 및 배포 금지

다음 행동을 알려 주는 에러 메시지

에러 메시지는 모델이 읽는 피드백이다

나쁜 메시지와 좋은 메시지

나쁜 메시지	좋은 메시지
KeyError: '123'	ticket_id는 TCK- 접두어와 숫자 6자리여야 합니다. 숫자만 받았다면 TCK-000123으로 다시 호출하세요.
ValidationError	new_status=resolved이면 resolution_note가 필요합니다. 해결 방법과 근거 doc_id를 1~3문장으로 적어 다시 호출하세요.
Timeout	docs_search가 3초 안에 끝나지 않아 중단했습니다. page_size를 줄이거나 검색어를 구체화해 다시 호출하세요. 같은 입력으로 곧바로 반복하지 마세요.

1. **무엇이 틀렸는지** 밝힘
2. **어떻게 고치는지** 고친 값 예시까지 제시
3. **고칠 수 없을 때** 사용자에게 묻거나 사람에게 넘김
4. 검증 오류도 정리: 내부 타입 표기와 None 분기 오류 제거(12번 슬라이드)

무단전재 및 배포 금지

예제 도구 재설계: 6개에서 3개로

API 래퍼 6개를 워크플로 도구 3개로 정리한다

재설계 전후 대응

새 도구	기존 도구	읽기 전용	핵심 변화
docs_search	list_docs, get_doc	예	query 검색 후 doc_ids로 상세
ticket_lookup	list_tickets, get_ticket	예	번호 또는 이메일 필수, 페이지
ticket_update	update_ticket	아니요	closed 불가, 해결 메모 필수
(제거)	raw_sql	-	범용 SQL은 최소 권한 위반

1. 합칠 때 잃는 기능이 없는지 확인함
2. doc_ids를 빠뜨리면 골라 읽기를 실행할 방법이 없음
3. 설명이 권하는 사용법은 파라미터로 실행 가능해야 함
4. 권한 통제는 6장에서 다룸

무단전재 및 배포 금지

poka-yoke를 입력 모델로 구현

잘못된 행동은 스키마와 검증기로 표현할 수 없게 한다

python

```
class TicketUpdateInput(BaseModel):
    model_config = ConfigDict(extra="forbid")
    ticket_id: TicketId          # TCK-000123으로 정규화
    comment: str = Field(min_length=5, max_length=2000)
    # closed는 enum에 없음: 종료는 사람만 한다
    new_status: Literal["pending", "resolved"] | None = None
    resolution_note: str | None = None

    @model_validator(mode="after")
    def resolved_needs_note(self):
        if (self.new_status == "resolved"
                and not self.resolution_note):
            raise ValueError("resolution_note가 필요합니다...")
        return self
```

1. TicketId는 AfterValidator로 대소문자를 정규화하고 숫자만 오면 올바른 형식을 안내
2. **extra="forbid"**로 모르는 필드를 거부
3. 필드 사이 규칙은 model_validator가 검사
4. ticket_lookup은 ticket_id 또는 requester_email이 없으면 거부

출처: 교재 harness/tools/models.py 발췌(Field 설명 생략)

무단전재 및 배포 금지

@beta_tool로 빠르게 정의하기

타입 힌트와 docstring에서 입력 스키마를 만든다

python

```
@beta_tool
def docs_search(query: str | None = None,
                doc_ids: list[str] | None = None,
                page: int = 1) -> str:
    """사내 문서를 검색하거나 지정 문서를 자세히 읽습니다.
    티켓 상태 확인에는 ticket_lookup을 쓰세요.
    """
    try:  # 스키마와 별개로 pydantic 재검증
        args = DocsSearchInput(query=query, doc_ids=doc_ids,
                               page=page)
    except ValidationError as e:
        raise ToolError(_validation_message(e)) from e
    return impl.docs_search(args)
```

1. client.beta.messages.tool_runner가 2장의 루프를 대신 돌림
2. 프로토타입에는 가장 빠른 방법
3. 타임아웃, 출력 상한, 감사 로그를 함수마다 반복해야 함
4. ToolError가 is_error로 바뀌는지는 ANTHROPIC_LOG=debug로 SDK 버전별 확인

출처: 교재 beta_tool_demo.py 발췌(response_format 인자와 Args 설명 생략)

무단전재 및 배포 금지

strict와 pydantic 재검증

strict는 스키마에 맞는 JSON까지만 보장한다

strict

- 도구 입력이 스키마에 맞는 유효한 JSON임을 보장
- 객체마다 additionalProperties: false 지정
- minimum/maximum, minLength/maxLength, minItems/maxItems 미지원
- strictify가 미지원 키워드와 기본값을 설명문으로 옮기고 title은 삭제
- 지원하지 않는 모델에 strict를 보내면 400 오류
- 지원 모델, 도구 수 제한, 베타 헤더는 적용 전 structured outputs 문서로 확인

재검증

1. 길이와 범위 제약은 pydantic이 실제로 검사
2. resolved면 resolution_note 필수 같은 필드 간 규칙
3. 티켓 번호 정규화처럼 스키마로 못 쓰는 처리
4. strict가 꺼진 경로(다른 SDK, 테스트)의 입력

> ***입력 검증의 최종 책임은 하네스 코드가 진다***

출처: Anthropic, Structured outputs (Claude Platform Docs)

무단전재 및 배포 금지

두 정의 경로: @beta_tool vs ToolSpec

프로토타입은 @beta_tool, 운영 하네스는 ToolSpec 레지스트리로

도구 정의 경로 비교

구분	@beta_tool(프로토타입)	ToolSpec.to_api()(운영)
스키마 출처	타입 힌트와 docstring	pydantic input_model
strict 적용	옵션 미확인 → to_dict()에 strictify와 strict 덧붙여 직접 전송	strictify 후 strict: true
범위와 교차 검증	함수 안에서 pydantic 재검증	실행기가 input_model로 검증
루프	tool_runner(strict 덧붙이면 직접 루프)	2장 루프에서 실행기 호출
타임아웃, 상한, 로그	함수마다 직접 반복 구현	실행기 한곳에서 처리
실패 표시	SDK 버전별 처리 확인 필요	실행기가 is_error: true 부착

1. ToolSpec 필드: name, description, input_model, fn, read_only, timeout_s, strict
2. REGISTRY 하나로 Messages API와 Agent SDK 정의를 함께 생성
3. REGISTRY 순서 고정으로 프롬프트 캐시 보호(9장)

> ***정의는 레지스트리 하나, 정책은 실행기 한곳에***

무단전재 및 배포 금지

Agent SDK 도구 서버 만들기

@tool 핸들러가 공통 실행기를 호출해 검증, 타임아웃, 감사 로그를 공유

python

```
def as_sdk_tool(spec: ToolSpec, ctx: RunContext):
    @tool(spec.name, spec.description,
          to_strict_schema(spec.input_model),
          annotations=ToolAnnotations(
             readOnlyHint=spec.read_only))
    async def handler(args: dict) → dict:
        call_id = f"sdk_{uuid.uuid4().hex[:8]}"
        r = await execute_tool(spec.name, call_id, args,
                               REGISTRY, ctx)
        out = {"content": [{"type": "text",
                            "text": r["content"]}]}
        if r.get("is_error"): out["is_error"] = True
        return out
    return handler
```

1. create_sdk_mcp_server로 **인프로세스 MCP 서버** 생성
2. 도구 이름은 mcp__<서버>__<도구> 규칙(예: mcp__helpdesk__docs_search)
3. **readOnlyHint**는 병렬 실행 힌트일 뿐 보안 장치가 아님
4. allowed_tools는 자동 승인 목록일 뿐, 쓰기 통제는 disallowed_tools, can_use_tool, PreToolUse 훅(6장)

> ***readOnlyHint를 잘못 붙이면 쓰기는 그대로, 경쟁 상태만 생긴다***

출처: Anthropic, Claude Agent SDK reference (Python); How the agent loop works (Agent SDK)

무단전재 및 배포 금지

공통 실행기 execute_tool()

모든 도구 호출이 한곳을 지나며 실패를 모델용 결과로 바꾼다

python

```
async def execute_tool(name, tool_use_id, raw, reg, ctx):
    spec, status = reg.get(name), "ok"  # 미등록 처리 생략
    try:
        args = spec.input_model.model_validate(raw)
        text = await asyncio.wait_for(
            _invoke(spec.fn, args), timeout=spec.timeout_s)
    except ValidationError as e:  # 교재는 ToolError로 바꿔 다시 던짐
        status, text = "invalid_input", _validation_message(e)
    except TimeoutError:
        status, text = "timeout", "범위를 좁혀 다시 호출하세요"
    except ToolError as e: status, text = "error", str(e)
    except Exception: status, text = "error", "내부 오류 안내"
    content, art_id, path = _cap(text, ctx, tool_use_id)
    _audit(ctx, {...})  # 이후 is_error를 붙여 반환
```

1. 흐름: 도구 확인 → 입력 검증 → 실행(3초 제한) → 4KB 상한 → 감사 로그
2. _validation_message가 내부 타입 표기, none_required 오류, 'Value error, ' 접두어를 지움
3. 4KB 초과 시 앞뒤 1.5KB만 남기고 모델에는 artifact_id만, 서버 경로는 감사 로그에만
4. 동기 함수는 스레드를 멈출 수 없어 I/O에 타임아웃(7장), 읽기 전용 호출은 asyncio.gather로 병렬

> ***execute_tool()은 절대 예외를 던지지 않는다***

출처: 교재 harness/executor.py 요약(구조 단순화)

무단전재 및 배포 금지

실습 안내

도구 6개를 3개로 재설계하고 같은 질문 10개로 비교한다

1. **재설계 도구 구현**
   도구 3개를 ToolSpec(strict)과 Agent SDK @tool 버전으로 구현

2. **단위 테스트와 poka-yoke 확인**
   test_executor.py 10개 통과, check_models.py로 거부 메시지 확인

3. **비교 준비**
   legacy_tools.py와 eval/questions.jsonl 10문항 준비

4. **측정과 실행**
   count_tool_tokens.py로 정의 토큰 측정, compare.py 세 조건 실행

5. **결과 기록**
   성공률, 토큰, is_error, 오프로딩을 표로 정리, 감사 로그와 트랜스크립트 확인

> ***실패 문항은 점수보다 트랜스크립트를 먼저 읽는다***

무단전재 및 배포 금지

실습: 재설계 전후 비교 조건

토큰 차이에 섞인 두 효과를 세 조건으로 떼어 본다

비교 조건

조건	도구	실행기 상한	보려는 것
before-nocap	레거시 6개	끔	수만 토큰 덤프 재현
before-cap	레거시 6개	켬(4KB)	실행기 상한만의 효과
after	재설계 3개	켬(4KB)	ACI 재설계의 효과

1. nocap과 cap의 차이는 상한 효과, cap과 after의 차이는 설계 효과
2. after는 오프로딩 0건, 9번과 10번 뒤에도 티켓 상태 그대로
3. before-cap은 잘린 중간 때문에 성공률이 떨어질 수 있음
4. 5번은 레거시에서 구조적으로 실패 (assignee_id만 반환)

> ***실행기 상한은 도구 설계를 대신할 수 없다***

무단전재 및 배포 금지

핵심 정리

1. **도구는 모델을 위한 사용자 인터페이스다**
   설명이 권하는 사용법은 파라미터로 실행할 수 있어야 한다

2. **API를 감싸지 말고 워크플로 단위의 적은 도구로 묶는다**
   list_all보다 search, 저수준 ID보다 사람이 읽는 필드

3. **도구의 기본 응답 자체를 실행기 상한보다 작게 만든다**
   response_format, 페이지네이션, 바이트 예산

4. **에러 메시지에는 틀린 점, 고칠 방법, 대안을 담는다**
   스택 트레이스와 서버 경로는 감사 로그에만

5. **poka-yoke로 잘못된 행동을 표현할 수 없게 한다**
   enum에서 closed 제외, 조건부 필수 필드, 형식 정규화

6. **strict는 JSON 형식까지만 보장하므로 pydantic으로 재검증한다**
   운영 경로는 ToolSpec 레지스트리 하나로 정의

7. **readOnlyHint는 병렬 실행 힌트일 뿐 보안 장치가 아니다**
   allowed_tools도 내장 도구를 막지 않는다

8. **모든 호출은 예외를 던지지 않는 공통 실행기를 거친다**
   검증, 타임아웃, is_error, 출력 상한, 감사 로그를 한곳에서

무단전재 및 배포 금지